>_ MCMP113 · Week 9

One-Dimensional Arrays

Declaration, assignment and element access · many values under one name

Asst. Prof. Hasan Oğuz, PhD
Introduction to Basic Algorithms and Programming · Istanbul Okan University
Vocational School · 2026–2027 Fall

How to use these notes

These notes follow the Week 9 slides and add what does not fit on a slide: box diagrams of every array, trace tables, the thirteen example programs with real sample runs, the common mistakes, a glossary and graded exercises. Every listing is the tested program (or a marked excerpt of it) from the `code` folder (`09_01` ... `09_13`); open it in VS Code and run it while you read. Typed input in the sample runs is shown in **amber**. Golden rule for the whole week: *when in doubt, draw the boxes and write the indices under them*. Methods are still not used (they come in Week 12): every program consists of top-level statements.

Contents

1 Learning outcomes **2**

2 Why arrays? **2**

2.1 The memory model: a row of numbered boxes . . . 3

3 Declaring, creating and accessing arrays **3**

3.1 Declaration and creation . . . 3

3.2 Default values . . . 3

3.3 Initializer lists . . . 4

3.4 Element access: `a[i]` reads, `a[i] = …` writes . . . 4

3.5 Example 1 – array basics . . . 4

3.6 `IndexOutOfRangeException`: the box does not exist . . . 6

4 Traversing, reading and printing **6**

4.1 The standard `for` loop . . . 6

4.2 `foreach`: every value, no index . . . 7

4.3 Reading *N* values and printing an array . . . 7

4.4 Example 2 – grade statistics . . . 7

4.5 Example 3 – printing in reverse . . . 9

4.6 Example 4 – a lookup table replaces a `switch` . . . 9

4.7 Example 5 – monthly temperatures: neighbours `a[i - 1]` and `a[i]` . . . 10

5 More element types and patterns **11**

5.1 Arrays of any type; a string is almost a `char` array . . . 11
5.2 Example 6 – vowel counter . . . 11
5.3 Counting arrays: `counts[value]++` . . . 12
5.4 Parallel arrays . . . 13
5.5 Building an array from earlier elements . . . 14

6 Arrays are reference types: copy or second name? 15

7 Common mistakes 17

8 Summary 17

9 Glossary (English – Turkish) 18

10 Exercises 19
10.1 Check yourself: predict the output . . . 19
10.2 In-class exercises (pairs) . . . 19
10.3 Homework . . . 19
10.4 Extra practice . . . 20

11 References 20

>_ 1 Learning outcomes

After this week you should be able to:

1. explain why an array is needed when many values of the same type must be *kept* (not only summed);
2. draw an array as a row of numbered boxes, with indices 0 to `Length` − 1;
3. declare and create arrays with `new T[n]` and with initializer lists, and state the default values;
4. read and assign elements with `a[i]`, and explain when `IndexOutOfRangeException` occurs;
5. traverse an array with `for` and `foreach`, read N values from the keyboard and print an array;
6. use the patterns *lookup table*, *counting array*, *parallel arrays* and *build from earlier elements*;
7. index a `string` like a read-only `char` array;
8. tell a second name for the same array (`b = a`) from a real copy (`Clone()` or a copy loop).

>_ 2 Why arrays?

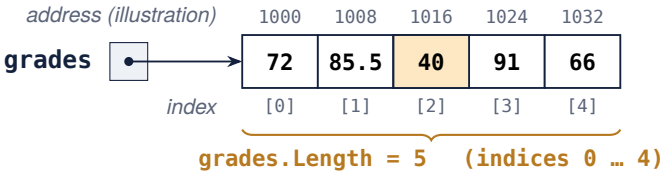
In Weeks 6–8 we computed the sum, the average, the minimum and the maximum of N inputs with a single variable `x` that was overwritten in every pass of the loop. Now try a slightly different task: *print how many grades are above the class average.* The average is known only after the last grade has been read — but by then the earlier grades are gone, because `x` only holds the last one. We must **keep** all values.

With separate variables (`g1`, `g2`, ...) this is hopeless: 100 students would need 100 names, and a loop cannot help, because a loop variable cannot select "the i-th name". An array solves exactly this problem.

Definition – array

An **array** is a fixed-size collection of values of the **same type**, stored side by side under **one name**. Each value is an ; it is selected by its , an `int` that counts from 0. An array of n elements has the indices $0, 1, \ldots, n-1$; the number n is its `Length`.

2.1 The memory model: a row of numbered boxes



- The variable `grades` does not contain the five numbers itself; it *refers to* a block of five `double` boxes that lie next to each other in memory (Section 6 shows why this matters).
- A `double` needs 8 bytes, so neighbouring boxes are 8 bytes apart. The addresses are only an illustration: C# chooses them for us and does not show them.
- The index says *how many boxes to skip from the start*. The first box skips none, so its index is 0; the last of five boxes has index $4 = \texttt{Length} - 1$. `grades[2]` is the third element; its value is 40.
- The size is fixed when the array is created. An array cannot grow or shrink (the growable `List<T>` is a topic for later courses).

>_ 3 Declaring, creating and accessing arrays

3.1 Declaration and creation

`int[] scores = new int[5];`	type `int[]` ("array of int"), name, `new` creates 5 boxes
`double[] grades = new double[n];`	the size may be any `int` expression, e.g. read at run time
`int[] scores; scores = new int[5];`	declaration and creation in two steps

The brackets belong to the *type*: `int scores[5];` (C/C++ style) does not compile in C#. A size of 0 is allowed (an empty array); a negative constant such as `new int[-1]` is a compile error (CS0248), and a negative size computed at run time stops the program with `OverflowException`.

3.2 Default values

Unlike a local variable (Week 3: using an unassigned local is error CS0165), every element of a new array is initialised automatically:

Element type	Default value	Example
`int`, `long`, `short`, `byte`	`0`	`new int[3]` → `0, 0, 0`
`double`, `float`, `decimal`	`0` (printed as `0`)	`new double[2]` → `0, 0`
`bool`	`false`	`new bool[2]` → `False, False`
`char`	`'\0'` (character code 0)	invisible when printed
`string`	`null` ("no string at all")	`null` ≠ `""` (the empty string)

3.3 Initializer lists

When the values are known while you write the program, list them in braces; the compiler counts them and sets the size:

```
int[] a = { 3, 7, 1 };                // short form: only in a declaration
int[] b = new int[] { 3, 7, 1 };      // with new and the type
var c = new[] { 3, 7, 1 };            // element type inferred: int[]
string[] days = { "Mon", "Tue", "Wed" };
char[] vowels = { 'a', 'e', 'i', 'o', 'u' };
```

All three of `a`, `b`, `c` have `Length` 3. `a.Length = 4;` does not compile (`Length` is read-only), and `a = new int[] { 9, 9 };` does not resize the old array: it creates a *new* two-element array and lets `a` refer to it.

3.4 Element access: `a[i]` reads, `a[i] = …` writes

An element behaves exactly like an ordinary variable: it can be read, assigned, incremented (`a[2]++`) and used in expressions. The index can be any `int` expression: `a[i]`, `a[i - 1]`, `a[a.Length - 1]` (the last element), even `a[a[0]]`.

3.5 Example 1 – array basics

Listing 1: 09_01_ArrayBasics/Program.cs

```
// MCMP113 · Week 9 · Example 1 — Array basics
// new int[5], default values, a[i], Length, initializers, for, foreach, string.Join.

// 1) Declaration: 5 int boxes, all start at the default value 0
int[] scores = new int[5];
Console.WriteLine($"Length: {scores.Length}");
Console.WriteLine($"After new int[5]: {string.Join(", ", scores)}");

// 2) Assignment and access with an index (first index 0, last index Length - 1)
scores[0] = 72;
scores[1] = 85;
scores[4] = 90;
scores[1] = scores[1] + 5;          // read a box, change it, write it back
Console.WriteLine($"After assignments: {string.Join(", ", scores)}");
Console.WriteLine($"First: {scores[0]}, last: {scores[scores.Length - 1]}");

// 3) Default values of other element types
double[] prices = new double[3];
bool[] present = new bool[3];
string[] names = new string[3];
char[] letters = new char[3];
Console.WriteLine($"double: {prices[0]}, bool: {present[0]}");
Console.WriteLine($"string is null: {names[0] == null}, char code: {(int)letters[0]}");

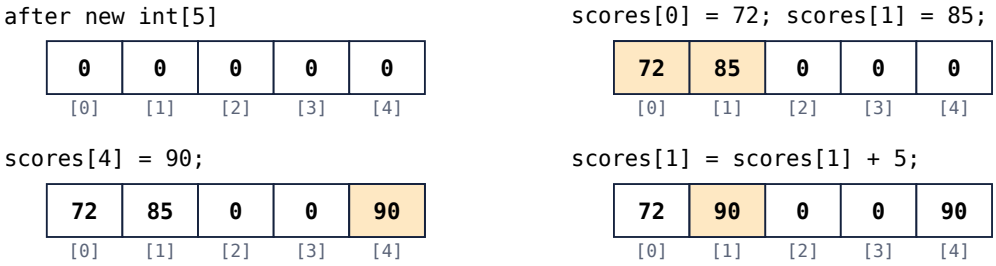
// 4) Initializer lists: size = number of values
```

```
26 int[] primes = { 2, 3, 5, 7, 11 };
27 string[] days = new[] { "Mon", "Tue", "Wed" };   // type inferred from the values
28
29 // 5) Traversal with for (index known) ...
30 for (int i = 0; i < primes.Length; i++)
31 {
32     Console.WriteLine($"primes[{i}] = {primes[i]}");
33 }
34
35 // ... and with foreach (only the values, read-only)
36 foreach (string day in days)
37 {
38     Console.Write(day + " ");
39 }
40 Console.WriteLine();
41
42 // 6) Printing: WriteLine(array) shows the TYPE name, not the values
43 Console.WriteLine(primes);
44 Console.WriteLine(string.Join(", ", primes));
45 for (int i = 0; i < primes.Length; i++)
46 {
47     Console.Write($"{primes[i],4}");      // width 4, right-aligned
48 }
49 Console.WriteLine();
```

Sample run (no input)

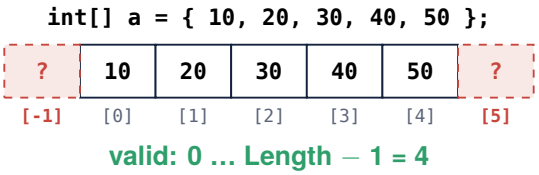
```
Length: 5
After new int[5]: 0, 0, 0, 0, 0
After assignments: 72, 90, 0, 0, 90
First: 72, last: 90
double: 0, bool: False
string is null: True, char code: 0
primes[0] = 2
primes[1] = 3
primes[2] = 5
primes[3] = 7
primes[4] = 11
Mon Tue Wed
System.Int32[]
2, 3, 5, 7, 11
   2   3   5   7  11
```

The boxes of `scores` after each group of assignments (lines 10–13):



Line 13 reads box 1 (85), adds 5 and writes 90 back into the same box. Line 43 shows a surprise: `Console.WriteLine(primes)` prints the *type name* `System.Int32[]` (`Int32` is the .NET name of `int`), not the values. Use `string.Join(", ", primes)` or a loop (lines 44–48).

3.6 `IndexOutOfRangeException`: the box does not exist



The compiler cannot know the value of a variable index, so a bad index is detected only *at run time*: the program stops with an unhandled `IndexOutOfRangeException`.

Listing 2: 09_03_OutOfRange/Program.cs

```
1  // MCMP113 · Week 9 · Example 3 — Valid and invalid indices
2  // Valid indices are 0 .. Length - 1; any other index throws IndexOutOfRangeException.
3
4  int[] a = { 10, 20, 30, 40, 50 };
5  Console.WriteLine($"Valid indices: 0 .. {a.Length - 1}");
6
7  Console.Write("Index: ");
8  int index = int.Parse(Console.ReadLine());
9
10 // Safe version: check the index first
11 if (index >= 0 && index < a.Length)
12 {
13     Console.WriteLine($"Checked:   a[{index}] = {a[index]}");
14 }
15 else
16 {
17     Console.WriteLine($"Checked:   {index} is not a valid index");
18 }
19
20 // Unsafe version: no check -> the runtime throws an exception for a bad index
21 Console.WriteLine($"Unchecked: a[{index}] = {a[index]}");
22 Console.WriteLine("Done.");
```

Run 1

```
Valid indices: 0 .. 4
Index: 3
Checked:   a[3] = 40
Unchecked: a[3] = 40
Done.
```

Run 2 – the program stops at line 21 (path shortened)

```
Valid indices: 0 .. 4
Index: 5
Checked:   5 is not a valid index
Unhandled exception. System.IndexOutOfRangeException:
 Index was outside the bounds of the array.
   at Program.<Main>$(String[] args) in
   .../09_03_OutOfRange/Program.cs:line 21
```

With the input -1 the output is the same except for the number. Read the message carefully: the exception type, the text and the **line number** tell you exactly where to look. The typical causes are a loop condition `i <= a.Length`, writing `a[a.Length]` for "the last element", and using a user's number $1..n$ as an index without subtracting 1. The cure is the check in lines 11–18: $0 \le$ `index` $<$ `a.Length`. (Catching exceptions with `try/catch` is a Week 14 topic.)

>_ 4 Traversing, reading and printing

4.1 The standard `for` loop

```
for (int i = 0; i < a.Length; i++)
```

```
{
    // work with a[i]
}
```

Start at 0 because the first index is 0; continue while `i < a.Length` (strictly less) because the last index is `Length` − 1. Always write `a.Length` instead of a literal such as 5: if the size changes, the loop still works. For `int[] primes = { 2, 3, 5, 7, 11 };` the loop of Example 1 (lines 30–33) runs as follows:

Test	i	i < 5	primes[i]	Output
1	0	true	2	`primes[0] = 2`
2	1	true	3	`primes[1] = 3`
3	2	true	5	`primes[2] = 5`
4	3	true	7	`primes[3] = 7`
5	4	true	11	`primes[4] = 11`
6	5	false	—	loop ends (`primes[5]` is never touched)

The condition is tested `Length` + 1 = 6 times; the body runs 5 times. With `i <= a.Length` the sixth test would be **true** and `primes[5]` would throw the exception.

4.2 `foreach`: every value, no index

```
foreach (string day in days)
{
    Console.Write(day + " ");      // Mon Tue Wed
}
```

`foreach` visits the elements in the order 0, 1, 2, . . . and gives you each *value* in the iteration variable. It is shorter and can never go out of range, but the iteration variable is **read-only**: `foreach (int x in a) { x = x * 2; }` is compile error **CS1656** ("Cannot assign to 'x' because it is a 'foreach iteration variable' "), and there is no index for labels or neighbours.

	for	foreach
index available	yes: `i`	no
can change elements	yes: `a[i] = …`	no (read-only)
neighbours `a[i - 1]`, `a[i + 1]`	yes	no
out-of-range risk	yes	no
use it to …	fill, change, compare	read every value (sum, count, print)

4.3 Reading N values and printing an array

Reading always follows the same three steps: read the size, create the array, fill it with one loop. Prompt with `i + 1` so that the user sees "Grade 1" while the program uses index 0. For printing there are three ways: a loop (full control over the format, e.g. `{x,6:F1}` columns), `string.Join(separator, array)` for a quick one-line list, and *not* `Console.WriteLine(array)`, which prints the type name. (Exception: a `char[]` passed to `WriteLine` is printed as text.)

4.4 Example 2 – grade statistics

Problem. Read n grades, print them, their average and how many are above the average.

Algorithm 1 – Above-average count

```
START
READ n;   grades ← new array of n numbers
FOR i ← 0  TO n − 1
    READ grades[i]
END FOR
sum ← 0
FOR i ← 0  TO n − 1
    sum ← sum + grades[i]
END FOR
average ← sum / n;   count ← 0
FOR i ← 0  TO n − 1
    IF grades[i] > average THEN
        count ← count + 1
    END IF
END FOR
PRINT average, count
END
```

Pass 3 is the reason for the array: it needs the average, which exists only after *all* grades have been summed.

Listing 3: 09_02_GradeStats/Program.cs (from line 4)

```
Console.Write("How many students? ");
int n = int.Parse(Console.ReadLine());

double[] grades = new double[n];

// Pass 1: read and store every grade
for (int i = 0; i < n; i++)
{
    Console.Write($"Grade {i + 1}: ");
    grades[i] = double.Parse(Console.ReadLine());
}

// Pass 2: the sum and the average
double sum = 0;
for (int i = 0; i < grades.Length; i++)
{
    sum += grades[i];
}
double average = sum / grades.Length;

// Pass 3: count the grades above the average
int aboveCount = 0;
for (int i = 0; i < grades.Length; i++)
{
    if (grades[i] > average)
    {
        aboveCount++;
    }
}

Console.WriteLine($"Grades: {string.Join(" | ", grades)}");
Console.WriteLine($"Average: {average:F2}");
Console.WriteLine($"Above average: {aboveCount} of {n}");
```

```
Sample run

How many students? 5
Grade 1: 72
Grade 2: 85.5
Grade 3: 40
Grade 4: 91
Grade 5: 66
Grades: 72 | 85.5 | 40 | 91 | 66
Average: 70.90
Above average: 3 of 5
```

Check by hand: $72 + 85.5 + 40 + 91 + 66 = 354.5$ and $354.5/5 = 70.9$; above it are 72, 85.5 and 91. Good test cases: all grades equal (the count is 0, because the test is `>`, not `>=`) and $n = 1$.

4.5 Example 3 – printing in reverse

The first number typed must be printed last, so the numbers have to be stored first. The second loop starts at the **last index** `numbers.Length - 1` and runs while `i >= 0`, so that index 0 is included.

Listing 4: 09_04_ReversePrint/Program.cs (lines 14–19)

```
14 Console.Write("Reversed:");
15 for (int i = numbers.Length - 1; i >= 0; i--)    // last index is Length - 1
16 {
17     Console.Write($" {numbers[i]}");
18 }
19 Console.WriteLine();
```

```
Sample run

How many numbers? 5
Number 1: 4
Number 2: 8
Number 3: 15
Number 4: 16
Number 5: 23
Reversed: 23 16 15 8 4
```

i	i >= 0	numbers[i]	printed
4	true	23	23
3	true	16	16
2	true	15	15
1	true	8	8
0	true	4	4
−1	false	—	loop ends

4.6 Example 4 – a lookup table replaces a `switch`

In Week 5 a day number became a day name with a seven-case `switch`. With an array the names are *data* and the day number minus one is the index. `lessonHours` is a *parallel* array (Section 5.4): the same index describes the same day.

Listing 5: 09_05_DayNames/Program.cs (from line 4)

```
4  string[] dayNames = { "Monday", "Tuesday", "Wednesday", "Thursday",
5                        "Friday", "Saturday", "Sunday" };
6  int[] lessonHours = { 4, 6, 2, 6, 3, 0, 0 };     // parallel array: same index, same day
7
8  Console.Write("Day number (1-7): ");
9  int day = int.Parse(Console.ReadLine());
10
11 if (day < 1 || day > dayNames.Length)
12 {
13     Console.WriteLine("There is no such day.");
14 }
15 else
```

```
{
    int index = day - 1;                          // 1-based number -> 0-based index
    string name = dayNames[index];
    Console.WriteLine($"{name}: {lessonHours[index]} lesson hours");
}
```

Run 1

```
Day number (1-7): 3
Wednesday: 2 lesson hours
```

Run 2

```
Day number (1-7): 8
There is no such day.
```

The check uses `dayNames.Length`, so the number 7 is not repeated in the code. Day 7 (index 6, the last index) prints `Sunday: 0 lesson hours`; without the check, day 8 would crash and day 0 would use index -1.

4.7 Example 5 – monthly temperatures: neighbours `a[i - 1]` and `a[i]`

The change from one month to the next needs two neighbouring elements. The loop therefore starts at `i = 1`: with `i = 0`, `temps[i - 1]` would be `temps[-1]`. The last loop only reads values, so `foreach` is enough.

Listing 6: 09_06_MonthlyTemperatures/Program.cs (from line 4)

```
string[] months = { "Jan", "Feb", "Mar", "Apr", "May", "Jun",
                    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec" };
// Example data: monthly average temperatures in degrees C (similar to Istanbul)
double[] temps = { 6.1, 6.4, 8.2, 12.3, 17.0, 21.6,
                   24.0, 24.2, 20.6, 16.2, 11.8, 8.1 };

double sum = 0;
for (int i = 0; i < temps.Length; i++)
{
    sum += temps[i];
}
double yearAverage = sum / temps.Length;
Console.WriteLine($"Year average: {yearAverage:F2} C");

Console.WriteLine("Month   Temp   Change");
// January has no previous month: the loop starts at index 1
Console.WriteLine($"{months[0],-5}{temps[0],7:F1}{"-",9}");
for (int i = 1; i < temps.Length; i++)
{
    double change = temps[i] - temps[i - 1];
    Console.WriteLine($"{months[i],-5}{temps[i],7:F1}{change,9:F1}");
}

Console.Write("Warmer than average:");
foreach (double t in temps)
{
    if (t > yearAverage)
    {
        Console.Write($" {t:F1}");
    }
}
Console.WriteLine();
```

Sample run (no input)

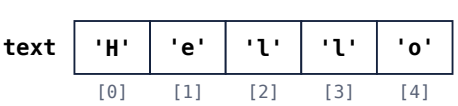
```
Year average: 14.71 C
Month   Temp   Change
Jan      6.1        -
Feb      6.4      0.3
Mar      8.2      1.8
Apr     12.3      4.1
May     17.0      4.7
Jun     21.6      4.6
Jul     24.0      2.4
Aug     24.2      0.2
Sep     20.6     -3.6
Oct     16.2     -4.4
Nov     11.8     -4.4
Dec      8.1     -3.7
Warmer than average: 17.0 21.6 24.0 24.2 20.6 16.2
```

Check: the twelve values add up to 176.5, and $176.5/12 = 14.708\overline{3} \approx 14.71$. The data are example values similar to Istanbul's monthly averages.

>_ 5 More element types and patterns

5.1 Arrays of any type; a string is almost a `char` array

Any type can be the element type: `double[]`, `string[]`, `char[]`, `bool[]` …; the loops stay the same. A `string` is not an array, but it can be *read* like one: `text[i]` is the character (a `char`) with index i and `text.Length` the number of characters, with the same index rules. For `string text = "Hello";`:



`text.Length` = 5, `text[0]` = `'H'`
`text[text.Length - 1]` = `'o'`
`text[5]` → exception

Strings are *immutable*: `s[0] = 'b';` is compile error **CS0200** ("Property or indexer 'string.this[int]' cannot be assigned to – it is read only").

5.2 Example 6 – vowel counter

The outer loop visits the characters of the text; the inner loop compares the current character with the five vowels. `vowelCounts` is parallel to `vowels`: `vowelCounts[v]` counts `vowels[v]`.

Listing 7: 09_09_VowelCounter/Program.cs (from line 4)

```
4  char[] vowels = { 'a', 'e', 'i', 'o', 'u' };
5  int[] vowelCounts = new int[vowels.Length];     // parallel to vowels[]
6
7  Console.Write("Text: ");
8  string text = Console.ReadLine();
9
10 int letters = 0;
11 for (int i = 0; i < text.Length; i++)
12 {
13     char c = char.ToLower(text[i]);
14     if (char.IsLetter(c))
15     {
```

```
16          letters++;
17      }
18      for (int v = 0; v < vowels.Length; v++)     // is c one of the vowels?
19      {
20          if (c == vowels[v])
21          {
22              vowelCounts[v]++;
23          }
24      }
25  }
26
27  int totalVowels = 0;
28  for (int v = 0; v < vowels.Length; v++)
29  {
30      Console.Write($"{vowels[v]}:{vowelCounts[v]}  ");
31      totalVowels += vowelCounts[v];
32  }
33  Console.WriteLine();
34  Console.WriteLine($"Letters: {letters}, vowels: {totalVowels}");
35  Console.WriteLine($"Consonants: {letters - totalVowels}");
36  if (text.Length > 0)                            // an empty text has no text[0]
37  {
38      char first = text[0];
39      char last = text[text.Length - 1];
40      Console.WriteLine($"First: '{first}', last: '{last}'");
41  }
```

Sample run

```
Text: Programming is fun!
a:1  e:0  i:2  o:1  u:1
Letters: 16, vowels: 5
Consonants: 11
First: 'P', last: '!'
```

`char.ToLower` makes `'A'` and `'a'` equal, `char.IsLetter` skips spaces and punctuation, so letters − vowels = consonants ($16 - 5 = 11$). The `if` in line 36 protects `text[0]` when the user just presses Enter (empty text, `Length` 0).

5.3 Counting arrays: `counts[value]++`

To count how often each of a small range of values occurs, use one counter per value in an array and let the *value* select the box. For a die, face 1..6 goes to index 0..5:

Listing 8: 09_07_DiceHistogram/Program.cs (lines 7–16)

```
7   // Fixed seed: the same rolls in every run (Random.Shared gives new rolls)
8   Random rng = new Random(2026);
9   // counts[0] = number of 1s, ..., counts[5] = number of 6s
10  int[] counts = new int[6];
11
12  for (int r = 0; r < rolls; r++)
13  {
14      int face = rng.Next(1, 7);      // 1..6 (7 is excluded)
15      counts[face - 1]++;             // face 1 -> index 0, face 6 -> index 5
16  }
```

For example rolls 3, 1, 3, 6 the counters change like this:

roll	index	[0]	[1]	[2]	[3]	[4]	[5]
—	—	0	0	0	0	0	0
3	2	0	0	**1**	0	0	0
1	0	**1**	0	1	0	0	0
3	2	1	0	**2**	0	0	0
6	5	1	0	2	0	0	**1**

The rest of the program (lines 18–29) prints one star per 1 %. The fixed seed `new Random(2026)` makes every run print the same rolls, so you can compare with these runs; `Random.Shared` gives new rolls each time.

```
60 rolls

How many rolls? 60
Face  Count  Share  Histogram (* = 1 %)
   1     16   26.7%  **************************
   2     12   20.0%  ********************
   3      3    5.0%  *****
   4      9   15.0%  ***************
   5      8   13.3%  *************
   6     12   20.0%  ********************
```

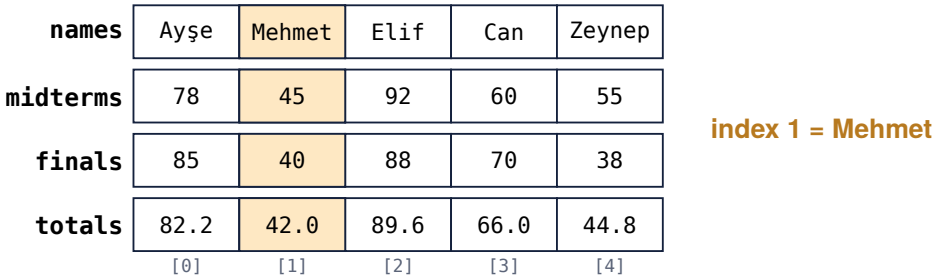
```
6000 rolls

How many rolls? 6000
Face  Count  Share  Histogram (* = 1 %)
   1   1037   17.3%  *****************
   2    967   16.1%  ****************
   3    981   16.4%  ****************
   4   1036   17.3%  *****************
   5    971   16.2%  ****************
   6   1008   16.8%  *****************
```

With 60 rolls the shares scatter widely (5 % to 26.7 %); with 6000 rolls all are close to $1/6 \approx 16.7\,\%$ (law of large numbers). Check: $16+12+3+9+8+12 = 60$ and $1037+967+981+1036+971+1008 = 6000$.

5.4 Parallel arrays

Parallel arrays are arrays of the same length in which the elements with the *same index* belong together.



Listing 9: 09_08_StudentReport/Program.cs (from line 4)

```
4 string[] names = { "Ayşe", "Mehmet", "Elif", "Can", "Zeynep" };
```

```
int[] midterms = { 78, 45, 92, 60, 55 };
int[] finals = { 85, 40, 88, 70, 38 };

double[] totals = new double[names.Length];
double classSum = 0;
for (int i = 0; i < names.Length; i++)
{
    totals[i] = 0.4 * midterms[i] + 0.6 * finals[i];   // MCMP113 weights: 40 % + 60 %
    classSum += totals[i];
}
double classAverage = classSum / names.Length;

Console.WriteLine($"{"Name",-8}{"Mid",5}{"Final",7}{"Total",8}  Result");
for (int i = 0; i < names.Length; i++)
{
    string result = totals[i] >= 50 ? "PASS" : "FAIL";   // example threshold
    Console.Write($"{names[i],-8}{midterms[i],5}{finals[i],7}");
    Console.WriteLine($"{totals[i],8:F1}  {result}");
}
Console.WriteLine($"Class average: {classAverage:F2}");
```

Sample run (no input)

```
Name      Mid  Final   Total  Result
Ayşe       78     85    82.2  PASS
Mehmet     45     40    42.0  FAIL
Elif       92     88    89.6  PASS
Can        60     70    66.0  PASS
Zeynep     55     38    44.8  FAIL
Class average: 64.92
```

Check: Ayşe $0.4 \cdot 78 + 0.6 \cdot 85 = 31.2 + 51 = 82.2$; class average $(82.2 + 42 + 89.6 + 66 + 44.8)/5 = 324.6/5 = 64.92$. The pass mark 50 is only an example threshold. **Danger:** if one array is sorted or an element is removed, all parallel arrays must be changed in exactly the same way, otherwise the data get mixed up (later courses group the data in a `class Student`).

5.5 Building an array from earlier elements

Some arrays are filled so that each element is computed from elements *before* it. The loop then starts after the "seed" elements that are set by hand.

Fibonacci numbers ($F_0 = 0$, $F_1 = 1$, $F_i = F_{i-1} + F_{i-2}$):

Listing 10: 09_11_FibonacciArray/Program.cs (lines 13–23)

```
    long[] fib = new long[n];
    fib[0] = 0;
    fib[1] = 1;
    // start at 2: the indices i - 1 and i - 2 already have values
    for (int i = 2; i < n; i++)
    {
        fib[i] = fib[i - 1] + fib[i - 2];
    }

    Console.WriteLine(string.Join(" ", fib));
    Console.WriteLine($"fib[{n - 1}] = {fib[n - 1]:N0}");
```

Sample runs

```
How many Fibonacci numbers (2-93)? 10
0 1 1 2 3 5 8 13 21 34
fib[9] = 34

How many Fibonacci numbers (2-93)? 94
Please choose a number from 2 to 93.
```

i	fib[i-2]	fib[i-1]	fib[i]
2	0	1	1
3	1	1	2
4	1	2	3
5	2	3	5
…			
9	13	21	34

The program accepts at most 93 numbers: the largest one, `fib[92]` $= 7\,540\,113\,804\,746\,346\,429$ (real output for $n = 93$), still fits in a `long` (maximum $\approx 9.22 \cdot 10^{18}$), but `fib[93]` would overflow.

Running total of a week's sales (`decimal` for money): `total[0] = sales[0]` is the seed, then `total`$[i]$ = `total`$[i-1]$ + `sales`$[i]$.

Listing 11: 09_12_RunningTotal/Program.cs (lines 14–25)

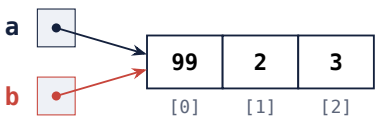
```
14 total[0] = sales[0];                    // the first day has nothing before it
15 for (int i = 1; i < days.Length; i++)
16 {
17     total[i] = total[i - 1] + sales[i];
18 }
19
20 Console.WriteLine("Day      Sales      Total");
21 for (int i = 0; i < days.Length; i++)
22 {
23     Console.WriteLine($"{days[i],-4}{sales[i],10:F2}{total[i],11:F2}");
24 }
25 Console.WriteLine($"Week total: {total[days.Length - 1]:F2} TL");
```

Sample run (inputs 1250, 980.50, 1430, 0, 2100.75, 3200, 1500 typed after the prompts "Sales on Mon (TL):" …)

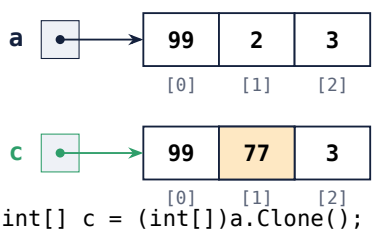
```
Day      Sales      Total
Mon    1250.00    1250.00
Tue     980.50    2230.50
Wed    1430.00    3660.50
Thu       0.00    3660.50
Fri    2100.75    5761.25
Sat    3200.00    8961.25
Sun    1500.00   10461.25
Week total: 10461.25 TL
```

>_ 6 Arrays are reference types: copy or second name?

An `int` variable holds its value directly (**value type**): `int y = x;` copies the number. An array variable holds only a **reference** to the boxes (**reference type**): `int[] b = a;` copies the reference, so `a` and `b` are *two names for one array*. A real copy needs a new block of boxes.



`int[] b = a; b[0] = 99;`
one array, two names



`int[] c = (int[])a.Clone(); c[1] = 77;`
a new array: only the copy changes

Listing 12: 09_10_CopyVsReference/Program.cs (from line 4)

```
4  int[] a = { 1, 2, 3 };
5
6  Console.WriteLine("1) int[] b = a;  b[0] = 99;");
7  int[] b = a;                    // NOT a copy: b refers to the same array
8  b[0] = 99;
9  Console.WriteLine($"   a = {string.Join(", ", a)}");     // a has changed too!
10 Console.WriteLine($"   b = {string.Join(", ", b)}");
11
12 Console.WriteLine("2) int[] c = (int[])a.Clone();  c[1] = 77;");
13 int[] c = (int[])a.Clone();      // a new array with the same values
14 c[1] = 77;
15 Console.WriteLine($"   a = {string.Join(", ", a)}");
16 Console.WriteLine($"   c = {string.Join(", ", c)}");
17
18 Console.WriteLine("3) copy loop into d;  d[2] = 55;");
19 int[] d = new int[a.Length];     // a new array, filled element by element
20 for (int i = 0; i < a.Length; i++)
21 {
22     d[i] = a[i];
23 }
24 d[2] = 55;
25 Console.WriteLine($"   a = {string.Join(", ", a)}");
26 Console.WriteLine($"   d = {string.Join(", ", d)}");
27
28 // == compares references (same array?), not the contents
29 Console.WriteLine($"a == b: {a == b},  a == c: {a == c}");
30
31 // int is a value type: assignment copies the value itself
32 int x = 1;
33 int y = x;
34 y = 99;
35 Console.WriteLine($"int: x = {x}, y = {y}");
```

Sample run (no input)

```
1) int[] b = a;  b[0] = 99;
   a = 99, 2, 3
   b = 99, 2, 3
2) int[] c = (int[])a.Clone();  c[1] = 77;
   a = 99, 2, 3
   c = 99, 77, 3
3) copy loop into d;  d[2] = 55;
   a = 99, 2, 3
   d = 99, 2, 55
a == b: True,  a == c: False
int: x = 1, y = 99
```

- `Clone()` returns an `object`, therefore the cast `(int[])` is needed.
- == on arrays compares *references* ("the same array?"), not contents: `a == c` is `False` although `a` and `c` once had equal values.
- Week 12 preview: passing an array to a method passes the reference, so the method can change the caller's elements.

>_ 7 Common mistakes

1. One pass too many: i <= a.Length

`for (int i = 0; i <= a.Length; i++)` uses `a[a.Length]` in the last pass → `IndexOutOfRangeException`. Use `i < a.Length`.

2. a[a.Length] as the last element

The last element is `a[a.Length - 1]`. A backward loop starts at `a.Length - 1` and runs while `i >= 0`.

3. Printing the array variable

`Console.WriteLine(a)` prints `System.Int32[]`. Use `string.Join(", ", a)` or a loop.

4. b = a as a copy

`int[] b = a;` gives a second name for the same array. Copy with `(int[])a.Clone()` or a loop into `new int[a.Length]`.

5. User numbers used as indices

Day 1..7 is index 0..6: subtract 1 and check the range first.

6. C/C++ syntax and foreach writes

`int a[5];` does not compile (write `int[] a = new int[5];`); assigning to a `foreach` variable is error CS1656; `a.Length = 10;` is not allowed (fixed size).

7. Neighbour access at the borders

`a[i - 1]` needs `i >= 1`, `a[i + 1]` needs `i <= a.Length - 2`: adjust the start or the end of the loop.

>_ 8 Summary

- An array stores many values of **one type** under **one name**; its size is fixed when it is created.
- Indices run from 0 to `Length` − 1; any other index throws `IndexOutOfRangeException` at run time.
- `new T[n]` fills the boxes with default values (0, `false`, `'\0'`, `null`); an initializer list `{ … }` sets the values and the size.
- `for (int i = 0; i < a.Length; i++)` to read or write elements; `foreach` only to read them. Print with `string.Join` or a loop.
- Patterns: lookup table (number → index → value), counting array (`counts[value]++`), parallel arrays (same index, same record), build from earlier elements (start the loop after the seeds).
- A string can be read like a `char` array: `text[i]`, `text.Length`, but not changed.
- `b = a` copies the reference (two names, one array); `Clone()` or a copy loop makes a new array. == compares references.

>_ 9 Glossary (English – Turkish)

English	Turkish	Meaning
array	dizi	fixed-size collection of values of one type
one-dimensional array	tek boyutlu dizi	elements in one row, one index
element	eleman (öğe)	one value (box) of an array
index	indis (sıra numarası)	position of an element, counted from 0
length	uzunluk	number of elements (`Length`)
declaration / creation	bildirim / oluşturma	`int[] a;` / `new int[5]`
initializer list	ilk değer listesi	`{ 3, 7, 1 }`
default value	varsayılan değer	value of a box after `new`
traverse (an array)	diziyi dolaşmak	visit every element in order
out of range / bounds	sınırların dışında	index < 0 or $\geq$ `Length`
exception	istisna (çalışma zamanı hatası)	run-time error object, e.g. `IndexOutOfRangeException`
lookup table	başvuru (arama) tablosu	array that maps a number to a value
counting (frequency) array	sayma (frekans) dizisi	one counter per possible value
histogram	histogram (sıklık grafiği)	bar chart of frequencies
parallel arrays	paralel diziler	same index = same record
running total	kümülatif (yürüyen) toplam	sum of all elements up to i
reference type / value type	başvuru tipi / değer tipi	variable holds a reference / the value
copy (clone)	kopya (klon)	new array with the same values
immutable	değiştirilemez	cannot be changed after creation (`string`)

>_ 10 Exercises

10.1 Check yourself: predict the output

Draw the boxes (and arrows), then write the exact output. The programs are the four quizzes of `09_13_PredictChecker`; run it to check.

Q1 – indices

```
int[] a = { 4, 8, 15, 16, 23 };
Console.WriteLine(a[1] + a[3]);
Console.WriteLine(a[a.Length - 1]);
Console.WriteLine(a[a[0] - 2]);
a[2] = a[0] * 2;
Console.WriteLine(string.Join(",", a));
```

Q2 – filling with a loop

```
int[] b = new int[5];
for (int i = 0; i < b.Length; i++)
{
    b[i] = i * i;
}
Console.WriteLine(b[4] - b[2]);
int s = 0;
for (int i = 0; i < b.Length; i += 2)
{
    s += b[i];
}
Console.WriteLine(s);
```

Q3 – two names, one array

```
int[] x = { 1, 2, 3 };
int[] y = x;
y[0] = 10;
int[] z = (int[])x.Clone();
z[1] = 20;
x[2] = 30;
Console.WriteLine($"{x[0]} {x[1]} {x[2]}");
Console.WriteLine($"{y[0]} {y[1]} {y[2]}");
Console.WriteLine($"{z[0]} {z[1]} {z[2]}");
```

Q4 – neighbours and counters

```
int[] p = { 1, 2, 3, 4, 5 };
for (int i = 1; i < p.Length; i++)
{
    p[i] = p[i - 1] + p[i];
}
Console.WriteLine(string.Join(" ", p));
int[] data = { 2, 0, 2, 3 };
int[] c = new int[4];
foreach (int v in data)
{
    c[v]++;
}
Console.WriteLine(string.Join(" ", c));
```

10.2 In-class exercises (pairs)

A. ⋆ **Sums.** Read 5 integers into an array; print the total and the sum of the elements with even index $(0, 2, 4)$.

B. ⋆ **Above the first.** Read 10 numbers; print every number that is larger than the first one.

C. ⋆⋆ **Palindrome.** Read a word and print whether it reads the same backwards. Compare `text[i]` with `text[text.Length - 1 - i]`.

D. ⋆⋆ **Two dice.** Roll two dice 1000 times, count the sums 2..12 in an `int[11]` and print a histogram.

E. ⋆⋆ **Seat booking.** `bool[] seats = new bool[10];` Reserve seat numbers 1..10 until 0 is typed (reject invalid and already reserved seats); then print the free seats.

F. ⋆ **Vector sum.** Read n and two arrays `a`, `b` of length n; build `c[i] = a[i] + b[i]` and print all three.

10.3 Homework

H1. ⋆ **Trace (paper).** `int[] a = new int[5];` and `a[i] = (i + 1) * (i + 1)` for $i = 0..4$; then print the elements from the last index to the first. Give the trace table and the output.

H2. ⋆ **Boxes and arrows (paper).** Draw the memory after `int[] p = {5, 6}; int[] q = p; q[1] = 0; int[] r = (int[])p.Clone(); r[0] = 9;`. What do `p`, `q`, `r` contain?

H3. ⋆ **Pseudocode (paper).** Write the pseudocode of exercise B with an array.

H4. ⋆⋆ **Week temperatures.** Read 7 daily temperatures (Mon ... Sun); print the average and the names of the days that were warmer than the average.

H5. ⋆⋆ **Turkish vowels.** Extend Example 6 to the eight Turkish vowels a, e, ı, i, o, ö, u, ü. Note that `char.ToLower('I')` gives `'i'` in the invariant culture, but in Turkish `I` belongs to `ı` and `İ` to `i`.

H6. ⋆⋆ **Week 10 preview.** Find the largest element of an array and its index. Start with `a[0]`, not with 0.

10.4 Extra practice

X1. ⋆ **Trace.** What is printed? Make a trace table with the columns `i`, `m[i]`, `k`.

```
int[] m = { 3, 1, 4, 1, 5, 9 };
int k = 0;
for (int i = 0; i < m.Length; i++)
{
    if (m[i] > m[k]) { k = i; }
}
Console.WriteLine($"{k} {m[k]}");
```

X2. ⋆ **Fill and filter.** Give the contents of `v` and the output.

```
int[] v = new int[6];
for (int i = 0; i < v.Length; i++) { v[i] = 10 - 2 * i; }
foreach (int t in v)
{
    if (t % 4 == 0) { Console.Write($"{t} "); }
}
```

X3. ⋆⋆ **Find the bugs.** The loop should print the elements with odd index from the last to the first, but it crashes. Find and fix the bug.

```
int[] e = { 10, 11, 12, 13, 14, 15, 16 };
for (int i = e.Length; i > 0; i -= 2) { Console.Write($"{e[i]} "); }
```

X4. ⋆⋆ **Reverse a char array.** Build the string that contains the characters of `char[] w = { 'k', 'a', 'y', 'a', 'k' };` in reverse order and print it with the length of `w`.

X5. ⋆⋆⋆ **Shift left.** Without a second array, move every element of an `int[]` one place to the left and put the old first element at the end (`{1, 2, 3, 4}` → `{2, 3, 4, 1}`).

>_ 11 References

- Microsoft Learn — Arrays (C# language reference): https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/arrays
- Microsoft Learn — Iteration statements (`for`, `foreach`, `do`, `while`): https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/iteration-statements

- Microsoft Learn — `IndexOutOfRangeException` class:
  https://learn.microsoft.com/en-us/dotnet/api/system.indexoutofrangeexception
- Microsoft Learn — `String.Join` method:
  https://learn.microsoft.com/en-us/dotnet/api/system.string.join
- Microsoft Learn — `Array.Clone` method:
  https://learn.microsoft.com/en-us/dotnet/api/system.array.clone
- Microsoft Learn — Value types and reference types:
  https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/value-types
  https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/reference-types
- Microsoft Learn — Default values of C# types:
  https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/default-values